

Assignments:

- AI Model Providers
- About Transformers

10 AI Model Providers — Models Released in Last 12 Months (Jul 2025 – Jul 2026)

#	Model Provider	Model	Release Date	Knowledge Cutoff	Cost (Input / Output per 1M tokens)	Context Window	Type	Best Use Cases
1	OpenAI	GPT-5.5	Apr 2026	Jan 2026	\$2.50 / \$10.00	512K tokens	Multimodal LLM	Complex reasoning, long-document analysis, enterprise coding, multimodal tasks
2	Anthropic	Claude Sonnet 5	Jun 2026	Feb 2026	\$3.00 / \$15.00 <i>(introductory until Aug 2026, then \$3/\$15)</i>	1M tokens	Multimodal LLM	Agentic workflows, software engineering, long-context tasks, enterprise use
3	Anthropic	Claude Opus 4.6	Feb 2026	Jan 2026	\$5.00 / \$25.00	1M tokens	Multimodal LLM	Highly complex reasoning, research, nuanced writing, deep analysis

#	Model Provider	Model	Release Date	Knowledge Cutoff	Cost (Input / Output per 1M tokens)	Context Window	Type	Best Use Cases
4	Google DeepMind	Gemini 2.5 Flash	Dec 2025	Oct 2025	\$0.15 / \$0.60	1M tokens	Multimodal LLM	Cost-efficient tasks, high-volume processing, fast inference, summarisation
5	Meta	Llama 4 Scout	Apr 2025	Mar 2025	Free / Open Source (self-hosted)	10M tokens	Open-weight LLM	On-premise deployment, privacy-sensitive apps, massive document processing
6	xAI	Grok 4.1	Jul 2025	Jun 2025	\$0.20 / \$0.80	2M tokens	Multimodal LLM	Real-time web search integration, long-context reasoning, research
7	DeepSeek	DeepSeek R1	Jan 2025	Nov 2024	\$0.55 / \$2.19	128K tokens	Reasoning LLM	Mathematics, code generation, step-by-step reasoning, cost-sensitive deployments
8	Mistral AI	Mistral Large 3	Mar 2026	Jan 2026	\$2.00 / \$6.00	128K tokens	Open-weight LLM	Enterprise tasks, multilingual, function

#	Model Provider	Model	Release Date	Knowledge Cutoff	Cost (Input / Output per 1M tokens)	Context Window	Type	Best Use Cases
9	Amazon (AWS)	Nova 2 Pro	Dec 2025	Sep 2025	~\$0.80 / \$3.20 (via Bedrock)	1M tokens	Multimodal LLM	calling, GDPR-compliant EU deployments
10	Alibaba (Qwen)	Qwen3-235B	May 2026	Mar 2026	\$0.01 / \$0.03	1M tokens (256K native + extrapolation)		AWS-native applications, enterprise automation, document understanding, agents

How Transformers Work

(a) Self-Attention Mechanism

One line: Every word looks at every other word and decides how much to "pay attention" to each one.

How it works:

Each word creates 3 vectors — **Q (Query)**, **K (Key)**, **V (Value)**

- Q = "What am I looking for?"
- K = "What do I offer to others?"

- $V = \text{"What is my actual content?"}$

Formula: $\text{Attention} = \text{softmax}(Q \times K^T / \sqrt{d}) \times V$

Example:

*"The animal didn't cross because **it** was tired"*

Self-attention helps the model link **"it"** → **"animal"** by scoring all word relationships and weighting them.

Multi-Head Attention = runs this process 8 times in parallel, each head catches different relationships (syntax, meaning, position).

(b) Vector Embedding

One line: Converts words into lists of numbers so the model can do math on them.

3 steps:

1. **Tokenise** — split text into tokens ("reading" → "read" + "ing")
2. **Embed** — map each token to a 512-number vector. Similar words get similar vectors ("king" ≈ "queen")
3. **Positional Encoding** — adds position info to each vector so the model knows word order

Why it matters: Without embeddings, the model cannot process language. Without positional encoding, "cat bit dog" and "dog bit cat" look identical.

(c) Feed-Forward Neural Network (FFN)

One line: After attention figures out word relationships, the FFN thinks deeper about each word individually.

Structure — 2 layers:

Input (512) → Layer 1 → Expand to 2048 → ReLU → Layer 2 → Back to 512

Key points:

- Runs **independently per token** (no cross-token interaction like attention)
- **Same FFN weights** applied to every token position
- **ReLU** adds non-linearity — enables learning complex patterns
- This is where **factual knowledge** is believed to be stored

